

GitHub Actions

Zaawansowane scenariusze CI/CD — 5 zadań

Rejestry kontenerów, dynamiczne macierze, optymalizacja przebiegów, ponowne użycie i automatyzacja wydań

Wymagania wstępne:

- Ukończona część podstawowa GitHub Actions (10 zadań)
- Konto GitHub z włączonymi Actions i GitHub Container Registry
- Znajomość Dockera (budowa obrazu) i podstaw monorepo

Zadania dotyczą wyłącznie warstwy CI/CD. Kod aplikacji (monorepo z serwisami web i api) otrzymujesz osobno — nie przepisuj go z tego dokumentu. Dockerfile oraz wszystkie workflow przygotowujesz samodzielnie.

Wprowadzenie teoretyczne

Część zaawansowana zakłada znajomość podstaw (workflow, joby, kroki, akcje, sekrety, artefakty, macierze, needs, środowiska). Poniżej skrót pięciu obszarów, których dotyczą zadania.

1. Rejestry kontenerów i GHCR

GitHub Container Registry (ghcr.io) to wbudowany rejestr obrazów Docker. Logowanie odbywa się wbudowanym tokenem **GITHUB_TOKEN** — bez zakładania dodatkowych sekretów — pod warunkiem nadania workflow uprawnień **permissions: packages: write**. W praktyce używa się trzech akcji: *docker/login-action* (logowanie), *docker/metadata-action* (automatyczne generowanie tagów i etykiet, np. z SHA, nazwy gałęzi, wersji semver) oraz *docker/build-push-action* (budowa i publikacja, z cache warstw *type=gha*).

2. Dynamiczne macierze

Macierz (**strategy.matrix**) zwykle definiuje się statycznie. Gdy zestaw wariantów ma wynikać z danych (pliku konfiguracyjnego, listy zmienionych modułów, odpowiedzi API), buduje się macierz **dynamicznie**: jeden job wylicza ją i zwraca jako **output** w formacie JSON, a kolejny konsumuje ją przez funkcję **fromJSON(...)**. W monorepo istotne jest wskazanie cache zależności dla właściwego katalogu (*cache-dependency-path*) oraz **fail-fast: false**, by awaria jednego wariantu nie zatrzymywała reszty.

3. Sterowanie przebiegiem: concurrency i filtrowanie ścieżek

concurrency grupuje przebiegi i z opcją **cancel-in-progress** anuluje starszy, niezakończony przebieg, gdy pojawi się nowszy — oszczędza minuty i skraca pętlę zwrotną. **Filtrowanie ścieżek** pozwala uruchamiać tylko joby dotyczące zmienionych części repozytorium: albo na poziomie wyzwalacza (*on.push.paths*), albo w jobie wykrywającym zmiany (np. akcja *dorny/paths-filter*), którego outputy bramkują kolejne joby przez **if**. Pominięty job ma status *skipped* i nie psuje wyniku workflow.

4. Composite actions a reużywalne workflowy

To dwa różne poziomy ponownego użycia. **Composite action** grupuje kilka **kroków** w jeden, wielokrotnie używalny krok (katalog *.github/actions/<nazwa>/action.yml, using: composite*); wywołujesz ją przez **uses** wewnątrz joba. **Reużywalny workflow** opakuje całe **joby** (wyzwalacz **workflow_call** z inputami, sekretami i outputami); wywołujesz go na poziomie joba przez **uses: ./.github/workflows/<plik>.yml**. Oba można łączyć: reużywalny workflow może w środku korzystać z composite action.

5. Automatyzacja wydań i bezpieczeństwo

Wydania wyzwała się zwykle **pushem tagu** (np. `v1.2.3`). Workflow buduje artefakty i tworzy **GitHub Release** (np. poleceniem `gh release create`) z załącznikami i automatycznymi notatkami. **Środowiska (environments)** pozwalają dodać reguły ochrony — np. wymóg ręcznej akceptacji (`required reviewers`) przed wdrożeniem. Zasada **najmniejszych uprawnień** (jawne *permissions* per workflow/job) oraz przypinanie wersji akcji to podstawy bezpieczeństwa pipeline'u; do uwierzytelniania w chmurze zamiast długożyjących kluczy stosuje się **OIDC**.

Mechanizm	Słowo kluczowe / akcja
Publikacja obrazu	permissions: packages: write, docker/login-action, docker/metadata-action, docker/build-push-action
Dynamiczna macierz	job outputs, fromJSON(...), strategy.matrix, fail-fast
Optymalizacja	concurrency + cancel-in-progress, dorny/paths-filter, if
Ponowne użycie	using: composite, on: workflow_call, inputs / outputs / secrets
Wydania	on.push.tags, gh release create, environment, permissions: contents: write

Zadanie 1: Publikacja obrazu Docker do GHCR

Cel

Zautomatyzuj budowę obrazu kontenera serwisu **api** i jego publikację w GitHub Container Registry, z automatycznym tagowaniem i cache'owaniem warstw.

Do wykonania

- ☐ Przygotuj własny **Dockerfile** dla serwisu *services/api* — obraz ma uruchamiać serwer na porcie 3000.
- ☐ Workflow uruchamiany przy pushu do **main** oraz przy pushu tagu **v***.
- ☐ Nadaj workflow **minimalne** uprawnienia umożliwiające publikację pakietu (odczyt repozytorium + zapis pakietów).
- ☐ Zaloguj się do **ghcr.io** wbudowanym tokenem — bez dodatkowych sekretów.
- ☐ Wygeneruj tagi obrazu automatycznie: skrót SHA, nazwa gałęzi, wersja semver (dla tagów) oraz **latest** dla gałęzi domyślnej.
- ☐ Zbuduj i wypchnij obraz z kontekstu *services/api*, wykorzystując cache warstw (GitHub Actions cache).
- ☐ Po przebiegu obraz ma być widoczny w sekcji **Packages** repozytorium.

Kryteria zaliczenia

- ✓ Workflow publikuje obraz do *ghcr.io/<owner>/<repo>/api*.
- ✓ Tagi obejmują skrót SHA oraz **latest** (dla main).
- ✓ Push tagu *v1.0.0* tworzy tag obrazu *1.0.0*.
- ✓ Uprawnienia workflow ograniczone do niezbędnego minimum.
- ✓ Kolejny przebieg korzysta z cache warstw (krótszy build).

Zadanie 2: Dynamiczna macierz testów

Cel

Zbuduj macierz testów generowaną dynamicznie w trakcie działania workflow, zamiast wpisywać warianty na sztywno.

Do wykonania

- ☐ Job przygotowujący ma odczytać definicję wariantów z pliku **matrix.json** i wystawić ją jako output.
- ☐ Job testujący ma zbudować macierz na podstawie tego outputu (parsowanie JSON).
- ☐ Każdy wariant uruchamia testy odpowiedniego serwisu na wskazanej wersji Node.
- ☐ Skonfiguruj cache zależności **osobno** dla każdego serwisu (ścieżka pliku lock zależna od wariantu).
- ☐ Awaria jednego wariantu nie może przerywać pozostałych.

Kryteria zaliczenia

- ✓ Skład macierzy wynika z **matrix.json**, a nie z treści workflow.
- ✓ Zmiana *matrix.json* zmienia zestaw zadań bez edycji workflow.
- ✓ Uruchamiają się trzy warianty (web@20, api@20, api@22), każdy zielony.
- ✓ Warianty są niezależne (brak przerywania po pierwszej awarii).

Zadanie 3: Filtrowanie ścieżek i kontrola współbieżności

Cel

Zoptymalizuj pipeline monorepo: uruchamiaj tylko joby dotyczące zmienionych części i anuluj nieaktualne przebiegi.

Do wykonania

- ☐ Skonfiguruj **concurrency**, tak aby nowy push na tę samą gałąź anulował poprzedni, niezakończony przebieg.
- ☐ Dodaj job wykrywający, które serwisy (**web**, **api**) uległy zmianie, i wystawiający tę informację jako outputy.
- ☐ Job budujący **web** uruchamiaj tylko, gdy zmieniono pliki w *services/web/*.
- ☐ Job testujący **api** uruchamiaj tylko, gdy zmieniono pliki w *services/api/*.
- ☐ Workflow ma reagować na **push** i **pull_request**.

Kryteria zaliczenia

- ✓ Zmiana wyłącznie w *services/web/* uruchamia job web, a job api jest pomijany.
- ✓ Zmiana w obu serwisach uruchamia oba joby.
- ✓ Szybki kolejny push anuluje wcześniejszy, trwający przebieg.
- ✓ Pominięte joby nie powodują niepowodzenia całego workflow.

Zadanie 4: Composite action i reużywalny workflow

Cel

Wydziel powtarzalną logikę CI na dwóch poziomach ponownego użycia: composite action (poziom kroków) oraz reużywalnego workflow (poziom jobów).

Do wykonania

- ☐ Utwórz lokalną **composite action** (w `.github/actions/`), która dla podanego serwisu i wersji Node instaluje zależności, uruchamia testy i zwraca **output** (np. czas trwania).
- ☐ Utwórz **reużywalny workflow** (`workflow_call`) przyjmujący `service` i `node-version`, korzystający z tej composite action i przekazujący jej output dalej jako własny output.
- ☐ Utwórz workflow nadrzędny wywołujący reużywalny workflow osobno dla **web** i **api** (api na innej wersji Node), a w osobnym jobie wypisujący zebrane outputy.

Kryteria zaliczenia

- ✓ Composite action wywoływana jest przez `uses` ze ścieżki lokalnej.
- ✓ Reużywalny workflow ma zadeklarowane inputy oraz output pochodzący z composite action.
- ✓ Workflow nadrzędny uruchamia oba serwisy i odczytuje ich outputy.
- ✓ Reużywalny workflow istnieje na gałęzi domyślnej, więc caller się rozwiązuje.

Zadanie 5: Automatyzacja wydania na tag

Cel

Zautomatyzuj wydanie: na podstawie pushu tagu zbuduj artefakt aplikacji i opublikuj GitHub Release, z wykorzystaniem środowiska.

Do wykonania

- ☐ Workflow uruchamiany przy pushu tagu zgodnego z **v***.
- ☐ Nadaj workflow uprawnienie pozwalające na utworzenie release (zapis zawartości).
- ☐ Zbuduj serwis **web** i spakuj zawartość *dist/* do archiwum.
- ☐ Utwórz **GitHub Release** dla danego tagu, dołącz archiwum jako załącznik i wygeneruj automatyczne notatki wydania.
- ☐ Przypisz jobowi środowisko **release** (docelowo z możliwością dodania reguł ochrony / ręcznej akceptacji).

Kryteria zaliczenia

- ✓ Push tagu *v0.1.0* tworzy Release *v0.1.0* z dołączonym archiwum.
- ✓ Workflow nie uruchamia się na zwykłym pushu — wyłącznie na tagu.
- ✓ Uprawnienia ograniczone do niezbędnych (zapis zawartości).
- ✓ Job jest powiązany ze środowiskiem **release**.

Środowisko i materiały

Do zadań otrzymujesz osobno paczkę startową z kodem aplikacji — **monorepo** o następującej strukturze:

- **services/web** — frontend Vite; *npm run build* tworzy *dist/*, *npm test* uruchamia testy (Vitest).
- **services/api** — serwer HTTP Node.js bez zależności; *npm test* uruchamia *node --test*, *npm start* wystawia */* i */health* na porcie 3000.
- **matrix.json** — konfiguracja wariantów do zadania 2.

Czego dokument celowo nie podaje: gotowych workflowów, Dockerfile ani fragmentów YAML. To są elementy do samodzielnego przygotowania — korzystaj z oficjalnej dokumentacji: <https://docs.github.com/actions>.

Wskazówki organizacyjne

- Pracuj w jednym repozytorium; każde zadanie to osobny plik w *.github/workflows/*.
- Dockerfile umieść w *services/api/*.
- Composite action umieść w *.github/actions/<nazwa>/action.yml*.
- Zadania 1 i 5 najlepiej testować na gałęzi **main** oraz przez tworzenie tagów.